

MCP Theory Questions

1. Difference Between a Tool, Resource, and Prompt in MCP

In MCP, a **tool** is something that allows the AI to perform an action or execute a function. I would use a tool when I want the MCP client to actually do something, such as searching my movie database or running a function to get information.

Example: If I create a movie MCP server, I could use a tool like `search_movies` when I want the MCP client to search for a particular movie.

A **resource** is data or information that an MCP server makes available to the AI. I would use a resource when the MCP client needs to read or access some existing information rather than perform an action.

Example: I could expose my collection of movie data as a resource so the MCP client can read information about the movies stored in it.

A **prompt** is a predefined instruction or template that helps guide how the AI should perform a particular task. I would use a prompt when I want to provide a reusable way of asking the AI to perform a specific type of task.

Example: I could create a prompt such as “Analyze this movie and explain its genre, themes, and main characters” and reuse it whenever I want to analyze a movie.

In simple terms, I think of them as:

- **Tool:** Do something.
- **Resource:** Give the AI some information.
- **Prompt:** Tell the AI how to perform a task.

2. Why Local MCP Servers Use stdio

A local MCP server can communicate with an MCP client using **stdio**, which stands for standard input and standard output. In practice, this means the MCP client starts the MCP server as a local process and communicates with it by sending messages through the process’s input and receiving responses through its output.

For example, if I have a local Python MCP server for my movie data, an MCP client such as Claude Desktop can start that server directly and send MCP requests to it through stdio. I do not need to run a separate HTTP server or expose an API endpoint on a port.

This makes sense for a local MCP server because both the MCP client and the MCP server are running on the same computer. Using stdio keeps the setup simple and avoids unnecessary network configuration such as ports, URLs, or HTTP routing.

For example, instead of running something like:

```
http://localhost:8000
```

the MCP client can directly start my MCP server and communicate with it through stdin and stdout.

So, for a local MCP server, stdio provides a simple way for the MCP client and the server to communicate without needing a traditional network-based API.